Practical 1: Conflation Algorithm

1. Title

Implementation of Conflation Algorithm to Generate a Document Representative of a Text File

2. Course Outcome

- Understand the concept of **conflation** in Information Retrieval.
- Generate a representative form of a document by reducing different word forms to a common form.

3. Aim & Objectives

Aim:

- To implement a **Conflation Algorithm** for generating a document representative of a given text file.

Objectives:

- To understand the concept of word conflation.
- To process the words present in a text document.
- To reduce different forms of words to a common form.
- To generate a simplified document representative.

4. Problem Statement

Implement a conflation algorithm to process the words in a given text file and convert related word forms into a common representation.
Generate and display the resulting document representative for use in an Information Retrieval system.

5. Prerequisites

- Basic knowledge of **Information Retrieval**.
- Basic understanding of text processing.
- Knowledge of strings and file handling.
- Basic programming knowledge.
- Understanding of words, terms, and document representation.

6. Hardware & Software Requirements

Hardware:

- Computer/Laptop.
- Minimum 4 GB RAM.

Software:

- Windows/Linux OS.
- Python 3.x.
- VS Code / PyCharm / any Python IDE.

7. Learning Outcome

After completing this practical, students will be able to:

- Understand the concept of **conflation**.
- Process text documents programmatically.
- Identify and combine related word forms.
- Generate a representative form of a document.

8. Algorithm

1. Start.
2. Read the input text file.
3. Convert all text into lowercase.
4. Extract individual words from the document.
5. Remove unwanted punctuation and symbols.
6. Apply the conflation technique to the words.
7. Combine related word forms into a common representation.
8. Store the processed terms.
9. Display the document representative.
10. Stop.

9. Program

```
import re


def conflation(text):
    # Convert text to lowercase
    text = text.lower()


    # Extract words
    words = re.findall(r'\b[a-z]+\b', text)
```

```
    representative = []

    for word in words:
        # Simple conflation rules
        if word.endswith("ies"):
            word = word[:-3] + "y"
        elif word.endswith("ing"):
            word = word[:-3]
        elif word.endswith("ed"):
            word = word[:-2]
        elif word.endswith("s") and not word.endswith("ss"):
            word = word[:-1]

        representative.append(word)

    return representative


# Read text file
with open("input.txt", "r") as file:
    text = file.read()

# Generate document representative
result = conflation(text)

print("Document Representative:")
print(" ".join(result))
```

10. Result

- The **Conflation Algorithm** was successfully implemented.
- The input text file was processed and a **document representative** was generated by converting related word forms into common forms.

11. Applications

- Document indexing.
- Search engines.
- Information retrieval systems.
- Text classification.
- Document similarity and matching.
- Search query processing.

12. Conclusion

- Thus, the Conflation Algorithm was successfully implemented to generate a simplified document representative from a text file. It helps reduce variations of words and improves the efficiency of information retrieval.

Practical 2: Document Retrieval Using Inverted Files

1. Title

Implementation of Document Retrieval Using Inverted Files

2. Course Outcome

- Understand the concept of **inverted files** in Information Retrieval.
- Implement efficient document retrieval using an **inverted index**.

3. Aim & Objectives

Aim:

- To implement a program for retrieving relevant documents using an **inverted file/index**.

Objectives:

- To understand the structure of an inverted index.
- To create an index of terms and documents.
- To search for a given term.
- To retrieve documents containing the searched term.

4. Problem Statement

Implement a program to create an inverted file from a collection of text documents and store the relationship between terms and documents. Use the inverted file to efficiently retrieve and display documents containing a given search term.

5. Prerequisites

- Basic knowledge of **Information Retrieval**.
- Understanding of documents and terms.
- Basic knowledge of strings and dictionaries.
- Basic programming and file-handling concepts.

6. Hardware & Software Requirements

Hardware:

- Computer/Laptop.
- Minimum 4 GB RAM.

Software:

- Windows/Linux OS.
- Python 3.x.
- VS Code / PyCharm / any Python IDE.

7. Learning Outcome

After completing this practical, students will be able to:

- Understand the concept of **inverted files**.
- Create an inverted index for documents.
- Search terms efficiently.
- Retrieve relevant documents from a collection.

8. Algorithm

1. Start.
2. Create a collection of text documents.
3. Read each document.
4. Convert the document text into lowercase.
5. Extract individual words.
6. Create an inverted index using **term → document list**.
7. Accept a search term from the user.
8. Search the term in the inverted index.
9. Retrieve the documents containing the term.
10. Display the retrieved documents.
11. Stop.

9. Program

```
import re


# Collection of documents
documents = {
    1: "Information retrieval is used for searching documents",
    2: "Search engines use information retrieval techniques",
    3: "Inverted files help in fast document retrieval"
}
```

```
# Create inverted index
inverted_index = {}

for doc_id, text in documents.items():
    words = re.findall(r'\b[a-z]+\b', text.lower())

    for word in words:
        if word not in inverted_index:
            inverted_index[word] = []

        if doc_id not in inverted_index[word]:
            inverted_index[word].append(doc_id)

# Display inverted index
print("Inverted Index:")
for term, doc_list in inverted_index.items():
    print(term, "->", doc_list)

# Search for a term
query = input("\nEnter search term: ").lower()

if query in inverted_index:
    print("Retrieved Documents:")
    for doc_id in inverted_index[query]:
        print("Document", doc_id, ":", documents[doc_id])
else:
    print("No document found.")
```

10. Result

- The **inverted file/index** was successfully created.

- The program successfully retrieved and displayed documents containing the searched term.

11. Applications

- Search engines.
- Digital libraries.
- Document search systems.
- Web search and indexing.
- Information retrieval systems.
- Database and file searching.

12. Conclusion

- Thus, a program for document retrieval using inverted files was successfully implemented. The inverted index provides an efficient way to locate and retrieve documents containing a particular search term

Practical 3: Precision and Recall

1. Title

Implementation of a Program to Calculate Precision and Recall for Sample Input

2. Course Outcome

- Understand the concepts of **Precision and Recall** in Information Retrieval.
- Evaluate the performance of an Information Retrieval system.

3. Aim & Objectives

Aim:

- To implement a program to calculate **Precision and Recall** for a given query and sample document sets.

Objectives:

- To understand Precision and Recall measures.
- To identify relevant and retrieved documents.
- To calculate Precision and Recall using standard formulas.
- To evaluate the effectiveness of a retrieval system.

4. Problem Statement

Implement a program to calculate Precision and Recall for a sample query **q1**, given an answer set **A** and relevant documents **Rq1**.
The program should identify relevant retrieved documents and calculate Precision and Recall using standard formulas.

5. Prerequisites

- Basic knowledge of Information Retrieval.
- Understanding of relevant and retrieved documents.
- Knowledge of sets and set operations.
- Basic programming concepts.

6. Hardware & Software Requirements

Hardware:

- Computer/Laptop.
- Minimum 4 GB RAM.

Software:

- Operating System: Windows/Linux.

- Programming Language: Python 3.x.
- IDE: VS Code/PyCharm/any Python IDE.

7. Learning Outcome

After completing this practical, students will be able to:

- Understand Precision and Recall.
- Identify relevant retrieved documents.
- Calculate Precision and Recall.
- Evaluate the performance of an Information Retrieval system.

8. Algorithm

1. Start.
2. Define the answer set **A** containing retrieved documents.
3. Define the relevant document set **Rq1**.
4. Find the intersection of A and Rq1.
5. Count the relevant retrieved documents.
6. Calculate Precision.
7. Calculate Recall.
8. Display the Precision and Recall values.
9. Stop.

9. Program

```
# Answer set A - Retrieved documents
A = {"D1", "D2", "D3", "D4"}


# Relevant documents for query q1
Rq1 = {"D1", "D3", "D5", "D6"}


# Find relevant retrieved documents
relevant_retrieved = A.intersection(Rq1)


# Calculate Precision
precision = len(relevant_retrieved) / len(A)
```

```
# Calculate Recall
recall = len(relevant_retrieved) / len(Rq1)

# Display results
print("Answer Set A:", A)
print("Relevant Documents Rq1:", Rq1)
print("Relevant Retrieved Documents:", relevant_retrieved)

print("Precision =", precision)
print("Recall =", recall)
```

10. Result

- The program for calculating **Precision and Recall** was successfully implemented.
- **Precision = 0.50 (50%)**
- **Recall = 0.50 (50%)**

11. Applications

- Search engine evaluation.
- Document retrieval systems.
- Web search systems.
- Digital libraries.
- Information filtering.
- Text search systems.

12. Conclusion

- Thus, the program for calculating Precision and Recall was successfully implemented.
- Precision and Recall help to evaluate the effectiveness of an Information Retrieval system

Practical 4: Feature Extraction in 2D Color Images

1. Title

Implementation of Feature Extraction in 2D Color Images and Plotting of Feature Histogram

2. Course Outcome

- Understand the concept of **feature extraction** from images.
- Extract color-based features from a 2D color image and represent them using a histogram.

3. Aim & Objectives

Aim:

- To implement a program for extracting **color features** from a 2D color image and plot a histogram of the extracted features.

Objectives:

- To understand image feature extraction.
- To read and process a color image.
- To extract color features from the input image.
- To plot a histogram for the extracted features.

4. Problem Statement

Implement a program to extract color features from a given 2D color image using RGB color channels.
Plot the histogram of the extracted color features to analyze the distribution of pixel intensities in the image.

5. Prerequisites

- Basic knowledge of Image Processing.
- Basic understanding of 2D color images.
- Knowledge of RGB color channels.
- Basic Python programming concepts.
- Basic knowledge of histograms.

6. Hardware & Software Requirements

Hardware:

- Computer/Laptop.
- Minimum 4 GB RAM.

Software:

- Operating System: Windows/Linux.
- Programming Language: Python 3.x.
- Libraries: OpenCV, Matplotlib.
- IDE: VS Code/PyCharm/Jupyter Notebook.

7. Learning Outcome

After completing this practical, students will be able to:

- Understand image feature extraction techniques.
- Read and process 2D color images.
- Extract RGB color features.
- Generate and interpret image histograms.

8. Algorithm

1. Start.
2. Read the input color image.
3. Check whether the image is loaded successfully.
4. Convert the image from BGR to RGB format.
5. Separate the Red, Green, and Blue channels.
6. Calculate the histogram for each color channel.
7. Plot the histograms using suitable labels.
8. Display the input image and feature histograms.
9. Stop.

9. Program

```
import cv2
import matplotlib.pyplot as plt


# Read the input image
image = cv2.imread("input.jpg")


# Check if image is loaded
if image is None:
```

```
    print("Image not found!")
    exit()

# Convert BGR image to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Separate RGB channels
red = image_rgb[:, :, 0]
green = image_rgb[:, :, 1]
blue = image_rgb[:, :, 2]

# Display the input image
plt.figure(figsize=(8, 5))
plt.imshow(image_rgb)
plt.title("Input Color Image")
plt.axis("off")
plt.show()

# Plot RGB histograms
plt.figure(figsize=(8, 5))

plt.hist(red.ravel(), bins=256, alpha=0.5, label="Red")
plt.hist(green.ravel(), bins=256, alpha=0.5, label="Green")
plt.hist(blue.ravel(), bins=256, alpha=0.5, label="Blue")

plt.title("RGB Color Feature Histogram")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```

10. Result

- The color features of the input 2D image were successfully extracted using the **RGB color channels**.
- The **RGB histogram** was successfully plotted to represent the distribution of pixel intensities.

11. Applications

- Image classification.
- Object recognition.
- Image retrieval.
- Face detection and recognition.
- Medical image analysis.
- Computer vision systems.

12. Conclusion

Thus, the program for extracting color features from a 2D color image was successfully implemented. The RGB histogram was plotted to visualize the distribution of color intensities in the input image.

Practical 5: Web Crawler for E-Commerce Product Information

1. Title

Implementation of a Web Crawler to Extract Product Information and Links from an E-Commerce Website Using Python

2. Course Outcome

- Understand the concept and working of a **Web Crawler**.
- Extract useful product information and links from web pages using Python.

3. Aim & Objectives

Aim:

- To implement a Python-based web crawler for extracting product information and links from an e-commerce website.

Objectives:

- To understand the working of web crawlers.
- To fetch web page content using Python.
- To extract product names, prices, and links.
- To display the extracted information.

4. Problem Statement

Implement a web crawler in Python to access an e-commerce product page and extract relevant product information such as product name, price, and product links. Display the extracted information in a structured format for further processing or analysis.

5. Prerequisites

- Basic knowledge of Python programming.
- Understanding of HTML and web pages.
- Basic knowledge of HTTP requests.
- Knowledge of HTML tags and elements.
- Basic understanding of web scraping.

6. Hardware & Software Requirements

Hardware:

- Computer/Laptop.

- Minimum 4 GB RAM.
- Internet connection.

Software:

- Operating System: Windows/Linux.
- Programming Language: Python 3.x.
- Libraries: Requests, BeautifulSoup.
- IDE: VS Code/PyCharm/Jupyter Notebook.

7. Learning Outcome

After completing this practical, students will be able to:

- Understand the working of a web crawler.
- Fetch web page content using Python.
- Extract product information from HTML pages.
- Extract and display product links.
- Understand the basic process of web data collection.

8. Algorithm

1. Start.
2. Specify the URL of the target e-commerce webpage.
3. Send an HTTP request to the webpage.
4. Retrieve the HTML content of the webpage.
5. Parse the HTML using BeautifulSoup.
6. Identify product names, prices, and links.
7. Extract the required information.
8. Display the extracted product information.
9. Stop.

9. Program

```
import requests
from bs4 import BeautifulSoup
from urllib.parse import urljoin


# URL of the website
```

```
url = "https://example.com/products"

# Send request to the website
response = requests.get(
    url,
    headers={"User-Agent": "Mozilla/5.0"}
)

# Check response
if response.status_code == 200:

    # Parse HTML content
    soup = BeautifulSoup(response.text, "html.parser")

    # Find product containers
    products = soup.find_all("div", class_="product")

    print("Product Information:\n")

    for product in products:

        # Extract product name
        name = product.find("h2")

        # Extract product price
        price = product.find("span", class_="price")

        # Extract product link
        link = product.find("a", href=True)
```

```
        product_name = name.get_text(strip=True) if name else "N/A"
        product_price = price.get_text(strip=True) if price else "N/A"

        if link:
            product_link = urljoin(url, link["href"])
        else:
            product_link = "N/A"

        print("Product:", product_name)
        print("Price:", product_price)
        print("Link:", product_link)
        print("-" * 50)

else:
    print("Unable to access the webpage.")
```

Note: Use this code with a website you are permitted to crawl, and adjust the HTML selectors (product, h2, price) according to that site's structure. Do not bypass login, CAPTCHA, robots restrictions, or other access controls.

10. Result

- The web crawler was successfully implemented using Python.
- Product names, prices, and links were extracted from the target webpage and displayed.

11. Applications

- E-commerce product search.
- Price comparison systems.
- Product information collection.
- Search engine indexing.
- Market research.
- Web information retrieval.

12. Conclusion

Thus, a Python-based web crawler was successfully implemented to extract product information and links from an e-commerce webpage. The practical demonstrates the basic process of web crawling and information extraction from web pages.

Practical 6: Live Weather Report

1. Title

Implementation of a Python Program to Find the Live Weather Report of a Given City

2. Course Outcome

- Understand the use of **Web APIs** for retrieving real-time information.
- Develop a Python program to fetch and display live weather information.

3. Aim & Objectives

Aim:

- To implement a Python program to retrieve the live weather report of a given city.

Objectives:

- To understand the working of weather APIs.
- To accept a city name as input.
- To retrieve live weather data.
- To display temperature, wind speed, weather condition, and description.

4. Problem Statement

Implement a Python program to obtain the live weather report for a given city using a weather API.
The program should display the **temperature, wind speed, weather condition, and description** of the specified city.

5. Prerequisites

- Basic knowledge of Python programming.
- Understanding of APIs and HTTP requests.
- Basic knowledge of JSON data.
- Knowledge of Python dictionaries and functions.
- Internet connection.

6. Hardware & Software Requirements

Hardware:

- Computer/Laptop.
- Minimum 4 GB RAM.

- Internet connection.

Software:

- Operating System: Windows/Linux.
- Programming Language: Python 3.x.
- Library: Requests.
- Weather API: OpenWeatherMap API.
- IDE: VS Code/PyCharm/Jupyter Notebook.

7. Learning Outcome

After completing this practical, students will be able to:

- Understand the concept of Web APIs.
- Send requests to a weather API using Python.
- Process JSON weather data.
- Display live weather information for a given city.

8. Algorithm

1. Start.
2. Import the required Python library.
3. Accept the city name from the user.
4. Provide the API key and construct the API URL.
5. Send a request to the weather API.
6. Receive the weather data in JSON format.
7. Extract temperature, wind speed, weather, and description.
8. Display the weather information.
9. Stop.

9. Program

```
import requests


# OpenWeatherMap API key
API_KEY = "YOUR_API_KEY"


# Get city name
```

```
city = input("Enter city name: ")

# API URL
url = (
    f"https://api.openweathermap.org/data/2.5/weather"
    f"?q={city}&appid={API_KEY}&units=metric"
)

# Send request
response = requests.get(url)

# Check response
if response.status_code == 200:

    data = response.json()

    # Extract weather information
    temperature = data["main"]["temp"]
    wind_speed = data["wind"]["speed"]
    weather = data["weather"][0]["main"]
    description = data["weather"][0]["description"]

    # Display result
    print("\nLive Weather Report")
    print("-------------------------")
    print("City:", city)
    print("Temperature:", temperature, "°C")
    print("Wind Speed:", wind_speed, "m/s")
    print("Weather:", weather)
    print("Description:", description)
```

```
else:
    print("City not found or API request failed.")
```

Note: Replace YOUR_API_KEY with a valid API key from the weather API provider before running the program.

10. Result

- The Python program was successfully implemented to retrieve live weather information.
- The program displays the **temperature, wind speed, weather condition, and description** for the given city.

11. Applications

- Weather forecasting applications.
- Travel and tourism applications.
- Agriculture and farming systems.
- Smart home systems.
- Mobile weather applications.
- Real-time information systems.

12. Conclusion

Thus, a Python program was successfully implemented to retrieve and display the live weather report of a given city using a weather API. The practical demonstrates how real-time data can be accessed and processed using APIs